

TYBCA • SEMESTER COURSE

Introduction to Machine Learning & Python

Concepts, process, and the essential Python toolkit for data science.

SESSION OVERVIEW

Agenda

01 Machine learning vs traditional programming

03 Types of machine learning

05 Python libraries for data science

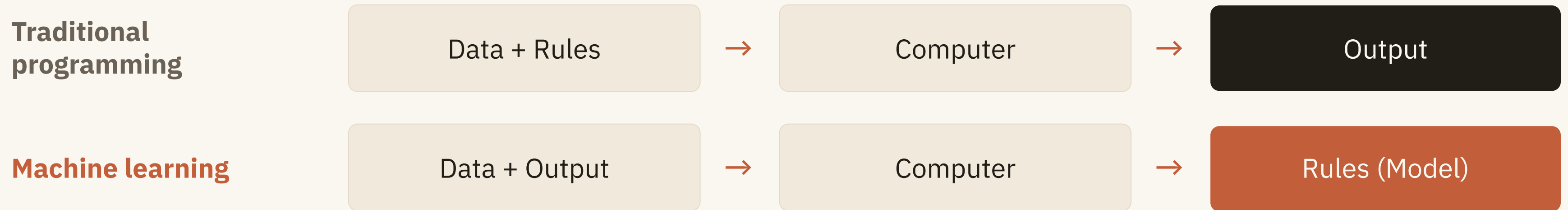
02 Steps of the machine learning process

04 Applications of machine learning

06 NumPy · Pandas · SciPy · Matplotlib

TOPIC 01

Machine Learning vs Traditional Programming



Example — a spam filter: instead of hand-coding rules for every spam word, we supply thousands of labelled emails and the algorithm learns the classification rules on its own.

A Side-by-Side Comparison

ASPECT	Traditional programming	Machine learning
Approach	Programmer writes explicit rules	Algorithm learns rules from data
Logic	Deterministic, fixed at design time	Statistical, inferred from patterns
Adaptability	Must be rewritten when rules change	Retrained on new data, improves over time
Data dependence	Works with little or no data	Requires large, quality datasets
Examples	Calculator, payroll system	Spam filter, recommendations, face recognition

TOPIC 02

Steps of the Machine Learning Process

1

Data collection

Gather raw data from databases, sensors, or the web

2

Data preparation

Clean, normalise, and split into training / test sets

3

Model selection

Choose an algorithm suited to the problem type

4

Training

Feed training data so the model learns its parameters

5

Evaluation

Measure accuracy on unseen test data

6

Tuning

Adjust hyperparameters to improve performance

7

Prediction

Deploy the model to make decisions on new data

Note

Steps 1–2 typically take 60–80% of project time

TOPIC 03

Types of Machine Learning

Supervised

Learns from labelled examples — every input has a known output

Feedback: correct answers provided

Unsupervised

Finds hidden structure in unlabelled data on its own

Feedback: none — patterns only

Semi-supervised

Combines a small labelled set with a large unlabelled one

Feedback: partial labels

Reinforcement

An agent learns by acting in an environment, by trial and error

Feedback: rewards and penalties

Supervised Learning

The model learns from labelled data — each training example pairs an input with its known output, like learning with a teacher who corrects mistakes.

- **Classification** — predicts a category: spam / not spam
- **Regression** — predicts a number: house price, temperature
- **Algorithms** — linear regression, decision trees, k-NN, SVM

EXAMPLE · HOUSE PRICE PREDICTION

Input: area, bedrooms, location

Label: actual sale price

Learned: $\text{price} = f(\text{area, bedrooms, ...})$

Use: predict the price of a new listing

Unsupervised Learning

The model receives unlabelled data and must discover hidden patterns, groupings, or structure on its own — no teacher, no correct answers.

- **Clustering** — grouping similar records: k-means, hierarchical
- **Dimensionality reduction** — compressing features: PCA
- **Association rules** — market-basket analysis: Apriori

EXAMPLE · CUSTOMER SEGMENTATION

Input: purchase history, no labels

Found: 3 natural groups of shoppers

Use: targeted offers per segment

Semi-Supervised Learning

A middle path: train with a small labelled dataset plus a large unlabelled one. Useful because labelling data is slow and expensive, while raw data is abundant.

- **Why** — expert labelling (doctors, linguists) is costly
- **How** — learn from labels, refine with pseudo-labelling
- **Where** — medical imaging, speech, web classification

EXAMPLE · MEDICAL SCANS

Labelled: 500 scans marked by radiologists

Unlabelled: 50,000 raw scans

Result: near-supervised accuracy at a fraction of the labelling cost

Reinforcement Learning

An agent learns by trial and error: it acts in an environment, receives rewards or penalties, and gradually learns a policy that maximises total reward.

- **Vocabulary** — agent, environment, state, action, reward, policy
- **Analogy** — training a dog with treats and corrections
- **Uses** — game playing (AlphaGo), robotics, self-driving

THE FEEDBACK LOOP

Agent takes an action



Environment returns new state + reward



Policy is updated — repeat

TOPIC 04

Applications of Machine Learning

SUPERVISED

Spam filtering · credit-risk scoring · medical diagnosis · weather forecasting

UNSUPERVISED

Customer segmentation · news topic grouping · fraud & anomaly detection

SEMI-SUPERVISED

Photo tagging · speech recognition · web page classification

REINFORCEMENT

Game AI (AlphaGo) · robotics · self-driving cars · recommendation ranking

Everyday touchpoints: UPI fraud alerts, OTT recommendations, voice assistants, maps ETA prediction.

Key Takeaways – Machine Learning Concepts

- 1 ML inverts traditional programming: it learns rules from data instead of executing hand-written rules.
- 2 Every ML project follows the same pipeline — collect, prepare, select, train, evaluate, tune, predict.
- 3 The four types differ by feedback: labels (supervised), none (unsupervised), partial (semi-supervised), rewards (reinforcement).
- 4 Choosing the right type starts with one question: what kind of data and feedback do we have?

TOPIC 05

Python Libraries for Data Science

Python dominates data science because of its readable syntax and a mature, free, open-source ecosystem built by a large community.

- Simple syntax — more time on the problem, less on the code
- Rich ecosystem — a library for every step of the pipeline
- Interoperable — libraries share NumPy arrays as a common format

Matplotlib

visualisation

Pandas

data analysis

SciPy

scientific computing

NumPy

the foundation — fast arrays

Pandas – Data Analysis

In computer programming, pandas is a software library written for the python programming language for data manipulation and analysis. In particular, it offers data structures and operations for manipulating numerical tables and time series

Pandas data structures: Series and Dataframe

- **Series** — one-dimensional labelled array
- **DataFrame** — 2-D table with named columns, like a spreadsheet
- **I/O** — read and write CSV, Excel, SQL, JSON
- **Cleaning** — filtering, grouping, missing-value handling

```
import pandas as pd

df = pd.read_csv("students.csv")
df.head()           # first 5 rows
df.describe()       # summary stats

toppers = df[df["marks"] > 75]
df = df.fillna(0)    # fix missing values
```

Functions:

- `pd.read_csv()`: Loads comma-separated value files into a DataFrame.
 - `pd.read_excel()`: Imports tabular data from Excel spreadsheets.
 - `pd.read_json()`: Reads JavaScript Object Notation data strings or files.
 - `df.to_csv()`: Writes the active DataFrame to a CSV format file.
 - `df.to_excel()`: Saves the DataFrame as an .xlsx workbook.
 - `df.head(n)`: Previews the first n rows of data (default is 5).
 - `df.tail(n)`: Previews the last n rows of data.
 - `df.info()`: Summarizes column names, missing data counts, and memory usage.
 - `df.describe()`: Computes descriptive statistics like mean, standard deviation, and quartiles.
 - `df.shape`: Returns an explicit count tuple representing total rows and columns.
 - `df.value_counts()`: Counts unique occurrences within a specified Series column.
-
- `df.isnull() / df.isna()`: Identifies missing or undefined NaN boundaries.
 - `df.fillna(value)`: Replaces missing numeric values with specific scalars or strategies.
 - `df.dropna()`: Strips entire rows or columns containing missing components.
 - `df.drop_duplicates()`: Sanitizes and discards repetitive duplicate rows.
 - `df.rename()`: Changes specific index labels or column names.
 - `df.astype()`: Converts or casts columns explicitly into specified data types.
-
- `df.drop('Marks', axis=1)`
 - `df.insert(1, "Gender", ['Female', 'Male', 'Male'])`
-
- `df.iloc[]`: Selects precise rows or columns based on integer positions.
 - example: `d1=df.iloc[0]` #fetch a row
 - `df.iloc[0:4]`
 - `df.iloc[0:4:2]`
 - `df.iloc[[1,3,6]]`
 - `df.iloc[:,[0,3]]`

NumPy – Numerical Computing

- **ndarray** — fast, fixed-type, n-dimensional array
- **Vectorised** — array maths runs in compiled C, 10–100× faster than loops
- **Broadcasting** — apply operations across whole arrays at once
- **Foundation** — Pandas, SciPy, and scikit-learn build on it

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])
print(arr * 2)          # [ 2  4  6  8 10]
print(arr.mean())       # 3.0

m = np.zeros((2, 3))    # 2x3 matrix of 0s
print(m.shape)          # (2, 3)
```

NumPy is a core Python library for numerical computing, built for handling large arrays and matrices efficiently. It is significantly faster than Python's built-in lists because it uses optimized C language style storage where actual values are stored at contiguous locations (not object reference).

- ndarray object: N-dimensional array for fast numerical operations.
- Vectorized operations: Perform element-wise calculations without loops.
- Broadcasting: Operate on arrays of different shapes.
- Linear algebra functions: Matrix operations like multiplication and inversion.
- Statistical functions: Mean, median, standard deviation.
- Integration: Works with Pandas, SciPy and scikit-learn.

- **Array Creation :** These functions allow you to initialize arrays from scratch or convert existing Python structures.
 - `np.array()`: Converts a Python list or tuple into a NumPy array.
 - `np.zeros()`: Creates an array filled entirely with zeros based on a given shape.
 - `np.ones()`: Generates an array filled entirely with ones.
 - `np.full()`: Creates an array of a specified size populated with a specific uniform value.
 - `np.arange()`: Returns evenly spaced values within a defined numerical range (similar to Python's `range`).
 - `np.linspace()`: Generates a specified number of evenly spaced values between two defined limits.
- **Array Inspection:** Use these properties and methods to extract structural metadata from an existing array.
 - `.shape`: Returns a tuple showing the number of elements along each dimension/axis.
 - `.ndim`: Displays the total number of dimensions (axes) the array possesses.
 - `.size`: Calculates the absolute total number of elements inside the array.
 - `.dtype`: Identifies the internal data type of the stored elements.
- **Array Manipulation:** These tools modify the structural layout or arrangement of data inside an array without altering its actual values.
 - `np.reshape()`: Changes the dimensions of an array without modifying its core data elements.
 - `.T` or `np.transpose()`: Flips the array's dimensions, swapping its rows with its columns.
 - `np.concatenate()`: Joins a sequence of separate arrays together along an existing axis.
 - `np.ravel()`: Flattens a multi-dimensional array down into a simple one-dimensional array.

- **Mathematical & Universal Functions (ufuncs)** : These functions apply mathematical operations element-wise across the entire array structure.
 - `np.add()` / `np.subtract()`: Performs element-by-element addition or subtraction.
 - `np.multiply()` / `np.divide()`: Performs element-by-element multiplication or division.
 - `np.sqrt()`: Computes the square root for every individual element in the array.
 - `np.exp()`: Calculates the exponential ((e^x)) for all elements.
 - `np.sin()` / `np.cos()`: Computes standard trigonometric ratios for values given in radians.
- **Statistical & Aggregation Functions**: These routines reduce data dimensions or extract statistical summaries from an array.
 - `np.sum()`: Totals all the elements together (can be constrained to rows or columns via the axis parameter).
 - `np.mean()`: Calculates the arithmetic average of the elements.
 - `np.median()`: Finds the middle value within the data distribution.
 - `np.std()` / `np.var()`: Evaluates the standard deviation and variance of the dataset.
 - `np.min()` / `np.max()`: Identifies the lowest or highest value contained in the array.
 - `np.argmin()` / `np.argmax()`: Returns the index position of the minimum or maximum value.

SciPy – Scientific Computing

- **Built on NumPy** — adds ready-made scientific algorithms
- **scipy.stats** — distributions, t-tests, correlation
- **scipy.optimize** — minimisation, curve fitting
- **scipy.linalg / integrate / signal** — linear algebra, integration, signals

```
from scipy import stats

a = [72, 75, 78, 80, 82]
b = [65, 68, 70, 72, 74]

t, p = stats.ttest_ind(a, b)
print(p)    # is the difference
            # statistically significant?
```

1. Optimization (**scipy.optimize**)

Used for finding function minima, maxima, and roots.

- **minimize()**: Finds the local minimum of a scalar or multivariate function.
- **root()**: Finds the roots (zeros) of continuous nonlinear equations.
- **curve_fit()**: Fits a user-defined function to a set of data points using non-linear least squares.

2. Numerical Integration (**scipy.integrate**)

Used to compute definite integrals and solve differential equations.

- **quad()**: Computes a definite integral of a one-variable function over a given range.
- **dblquad()**: Computes a double integral for two-variable equations.
- **solve_ivp()**: Solves initial value problems for ordinary differential equation (ODE) systems.

3. Linear Algebra (**scipy.linalg**)

Provides faster, more advanced matrix algorithms than standard NumPy by utilizing underlying LAPACK and BLAS libraries.

- **inv()**: Computes the inverse of a square matrix.
- **det()**: Calculates the determinant of a square matrix.
- **solve()**: Solves a linear system of equations ($Ax = b$) efficiently.
- **eig()**: Finds eigenvalues and eigenvectors of a square matrix.

4. Statistical Functions (**scipy.stats**)

Contains a vast collection of continuous and discrete probability distributions alongside statistical tests.

- **norm**: Sub-module for working with the normal (Gaussian) distribution, featuring methods like `.pdf()` (density), `.cdf()` (cumulative distribution), and `.rvs()` (random sampling).
- **ttest_ind()**: Conducts an independent two-sample t-test to compare group means.
- **pearsonr()**: Computes the Pearson correlation coefficient between two datasets.

5. Interpolation (**scipy.interpolate**)

Used to estimate unknown values that fall between known data points.

- **interp1d()**: Creates a linear or cubic 1D interpolation function based on existing data points.
- **griddata()**: Interpolates unstructured, multi-dimensional data onto a structured grid.

6. Signal Processing (**scipy.signal**)

Essential for analyzing, filtering, and manipulating continuous or discrete signals.

- **spectrogram()**: Computes a spectrogram (frequency vs. time) from a signal.
- **find_peaks()**: Locates local maxima (peaks) inside a 1D array based on physical criteria.
- **butter()**: Designs a digital or analog Butterworth filter.

Matplotlib – Data Visualisation

Matplotlib is an open-source library for creating static, animated and interactive visualizations in Python. Its object-oriented API enables the embedding of plots into applications developed with GUI toolkits such as Tkinter, Qt and GTK.

- It supports line plots, bar charts, histograms, scatter plots and 3D visualizations.
 - Provides full command over figure elements, including axes, labels, legends and colors.
 - Built to work with NumPy arrays and Pandas DataFrames.
 - Compatible with multiple general-purpose GUI toolkits (Tkinter, Qt, GTK and wxPython).
 - Capable of generating high-quality output in various hardcopy formats (PNG, PDF, SVG).
-
- **pyplot** — the standard plotting interface
 - **Line** — trends over time • **Bar** — comparisons
 - **Scatter** — relationships • **Histogram** — distributions
 - **Always label** — title, xlabel, ylabel on every chart
- Features of Matplotlib**
 - Versatile Plotting
 - Extensive Customization
 - Seamless Integration with NumPy
 - High-Quality Graphics
 - Cross-Platform Compatibility
 - Interactive Visualizations

Matplotlib Pyplot: Pyplot is a module within Matplotlib that provides a MATLAB-like interface for making plots. It simplifies the process of adding plot elements such as lines, images and text to the axes of the current figure.

Steps to Use Pyplot:

- **Import Matplotlib:** Start by importing matplotlib.pyplot as plt.
- **Create Data:** Prepare your data in the form of lists or arrays.
- **Plot Data:** Use plt.plot() to create the plot.
- **Customize Plot:** Add titles, labels and other elements using methods like plt.title(), plt.xlabel() and plt.ylabel().
- **Display Plot:** Use plt.show() to display the plot.

Line Chart: *matplotlib.pyplot.plot(x, y)*

```
import matplotlib.pyplot as plt

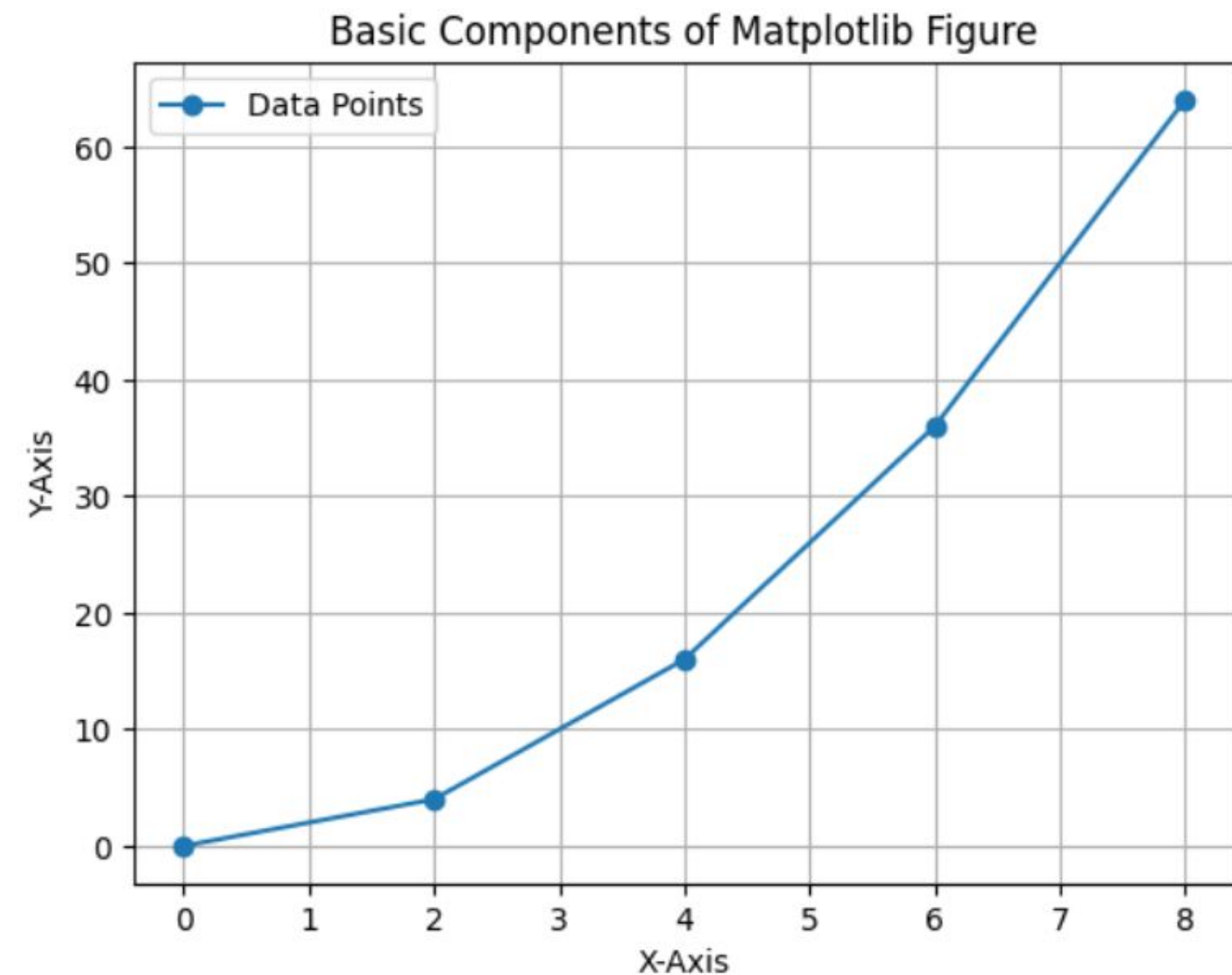
x = [0, 2, 4, 6, 8]
y = [0, 4, 16, 36, 64]

fig, ax = plt.subplots()

ax.plot(x, y, marker='o', label="Data Points")

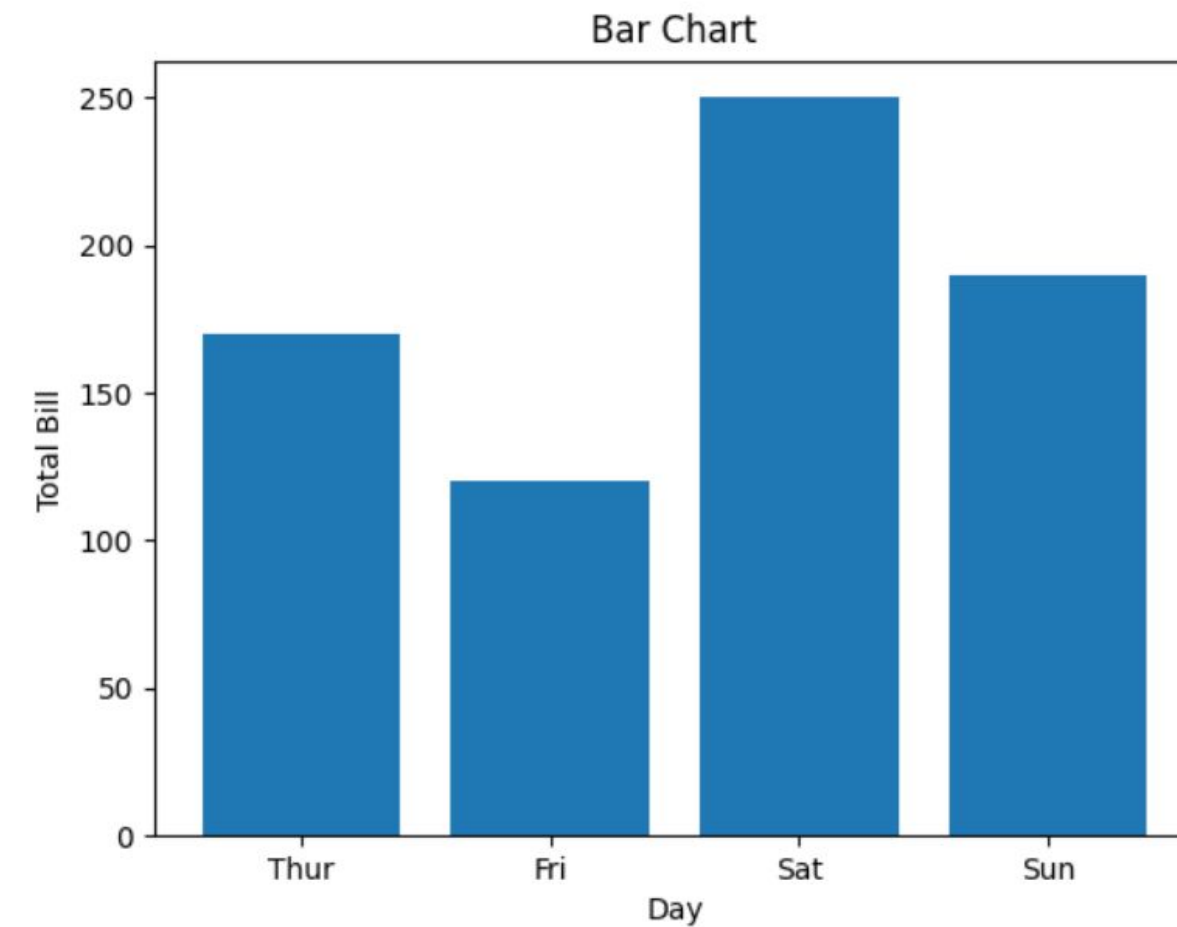
ax.set_title("Basic Components of Matplotlib Figure")
ax.set_xlabel("X-Axis")
ax.set_ylabel("Y-Axis")
ax.legend()

plt.show()
```



Bar Chart: `matplotlib.pyplot.bar(x, height)`

```
import matplotlib.pyplot as plt
x = ['Thur', 'Fri', 'Sat', 'Sun']
y = [170, 120, 250, 190]
plt.bar(x, y)
plt.title("Bar Chart")
plt.xlabel("Day")
plt.ylabel("Total Bill")
plt.show()
```



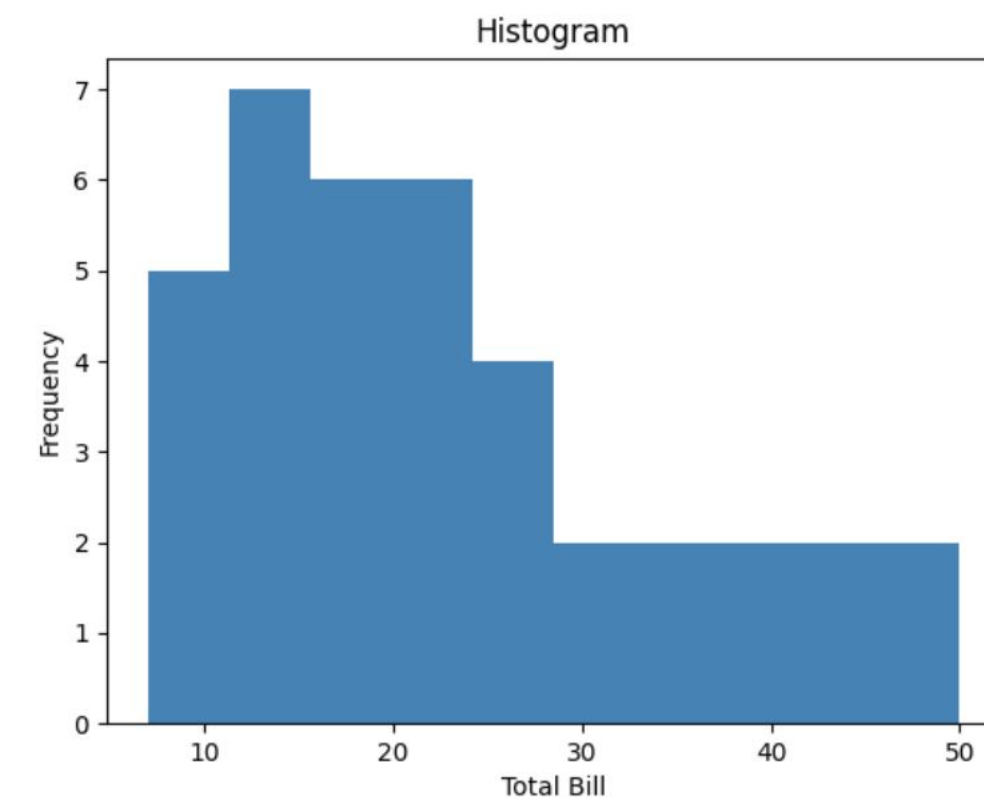
Histogram: `matplotlib.pyplot.hist(x, bins=None)`

Histogram shows the distribution of data by grouping values into bins. The `hist()` function is used to create it, with X-axis showing bins and Y-axis showing frequencies.

Example: This code plots a histogram to show frequency distribution of total bill values from the list `x`. It uses 10 bins and adds axis labels and a title for clarity.

Example:

```
import matplotlib.pyplot as plt
x = [7, 8, 9, 10, 10, 12, 12, 12, 13, 14, 14, 15, 16, 16, 17, 18, 18, 19, 20, 20,
     21, 22, 23, 24, 25, 25, 26, 28, 30, 32, 35, 36, 38, 40, 42, 44, 48, 50]
plt.hist(x, bins=10, color='steelblue')
plt.title("Histogram")
plt.xlabel("Total Bill")
plt.ylabel("Frequency")
plt.show()
```

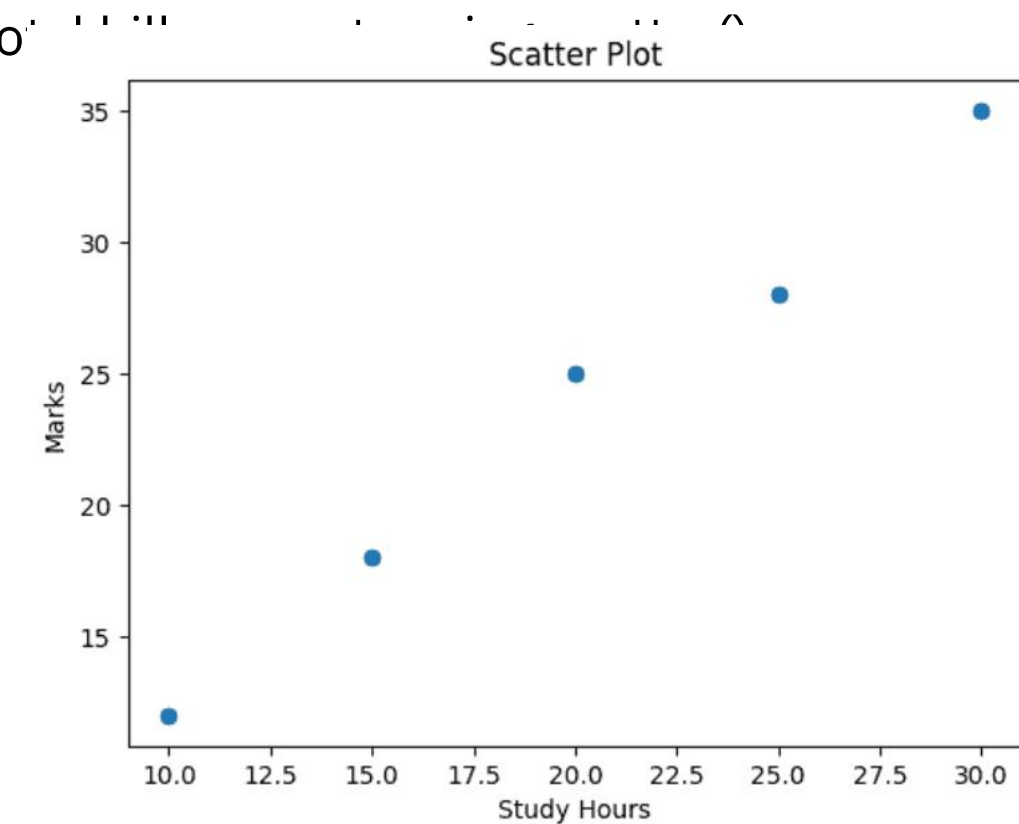


Scatter Plot : `matplotlib.pyplot.scatter(x, y)`

Scatter plots are used to observe relationships between variables. The `scatter()` method in the `matplotlib` library is used to draw a scatter plot.

Example: This code creates a scatter plot to visualize the relationship between days and to

```
import matplotlib.pyplot as plt
x = [10, 15, 20, 25, 30]
y = [12, 18, 25, 28, 35]
plt.scatter(x, y)
plt.title("Scatter Plot")
plt.xlabel("Study Hours")
plt.ylabel("Marks")
plt.show()
```



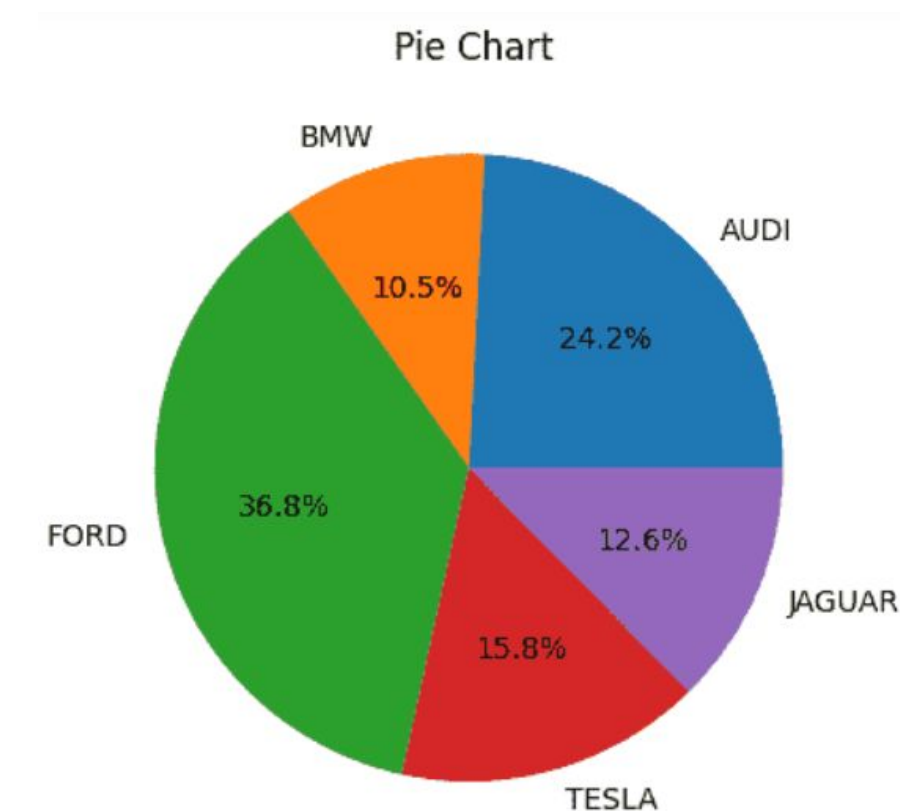
Pie Chart: `matplotlib.pyplot.pie(x, labels=None, autopct=None)`

Where x: Data values for pie slices.

labels: Names for each slice.

autopct: Format to display percentage (e.g., '%1.1f%%').

```
import matplotlib.pyplot as plt
cars = ['AUDI', 'BMW', 'FORD', 'TESLA', 'JAGUAR']
data = [23, 10, 35, 15, 12]
plt.pie(data, labels=cars, autopct='%1.1f%%')
plt.title(" Pie Chart")
plt.show()
```



Box Plot : Box plot is a simple graph that shows how data is spread out. It displays the minimum, maximum, median and quartiles and also helps to spot outliers easily.

```
matplotlib.pyplot.boxplot(x, notch=False, vert=True)
```

x: Data for which box plot is to be drawn (usually a list or array).

notch: If True, draws a notch to show the confidence interval around the median.

vert: If True, boxes are vertical. If False, they are horizontal.

Example:

```
import matplotlib.pyplot as plt
```

```
data = [ [10, 12, 14, 15, 18, 20, 22], [8, 9, 11, 13, 17, 19, 21],  [14, 16, 18, 20, 23, 25, 27] ]
```

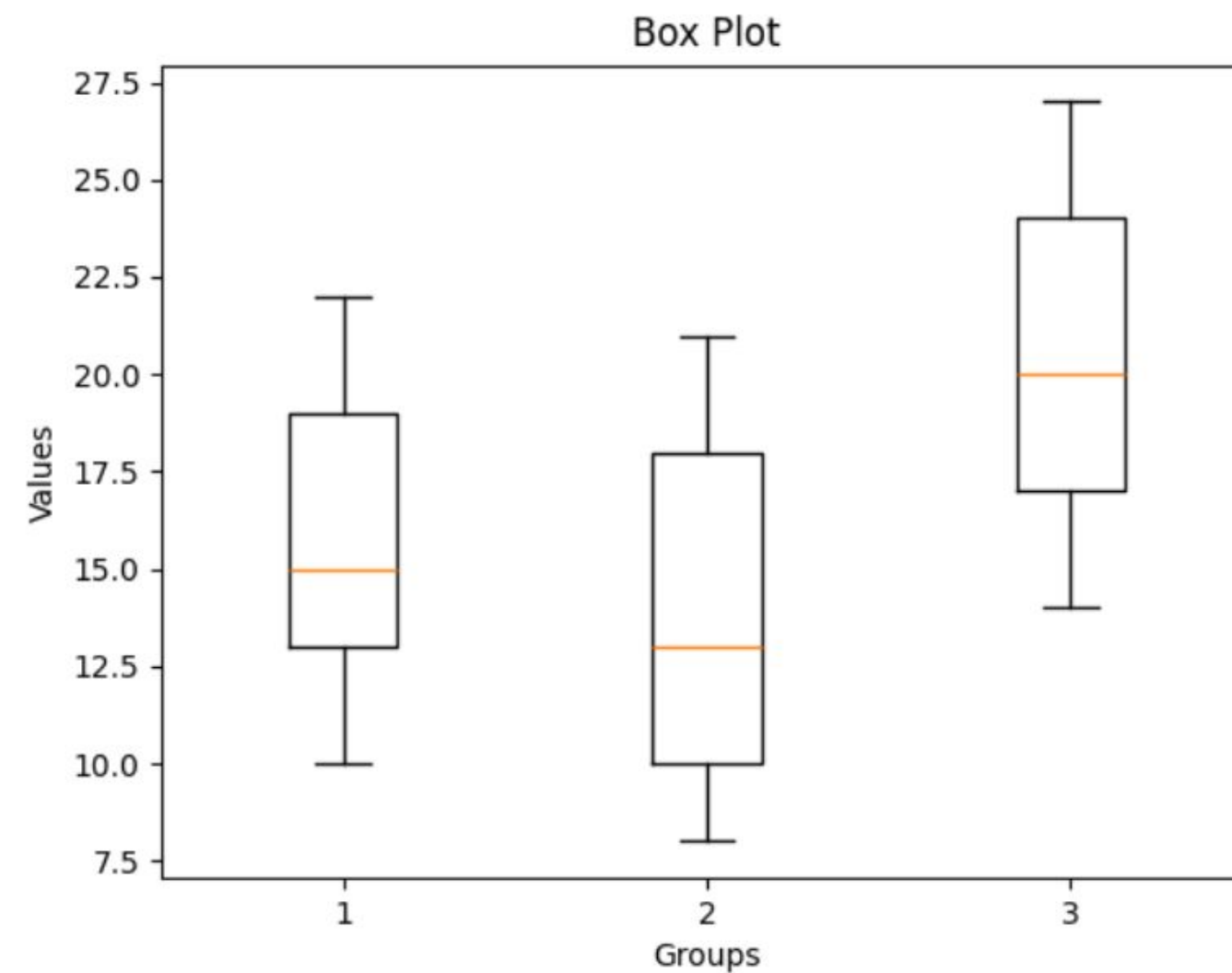
```
plt.boxplot(data)
```

```
plt.xlabel("Groups")
```

```
plt.ylabel("Values")
```

```
plt.title("Box Plot")
```

```
plt.show()
```



Heatmap : Heatmap is a graphical representation of data where values are shown as colors. It helps visualize patterns, correlations or intensity in a matrix-like format. It is created using **imshow()** method in Matplotlib.

```
matplotlib.pyplot.imshow(X, cmap='viridis')
```

where:

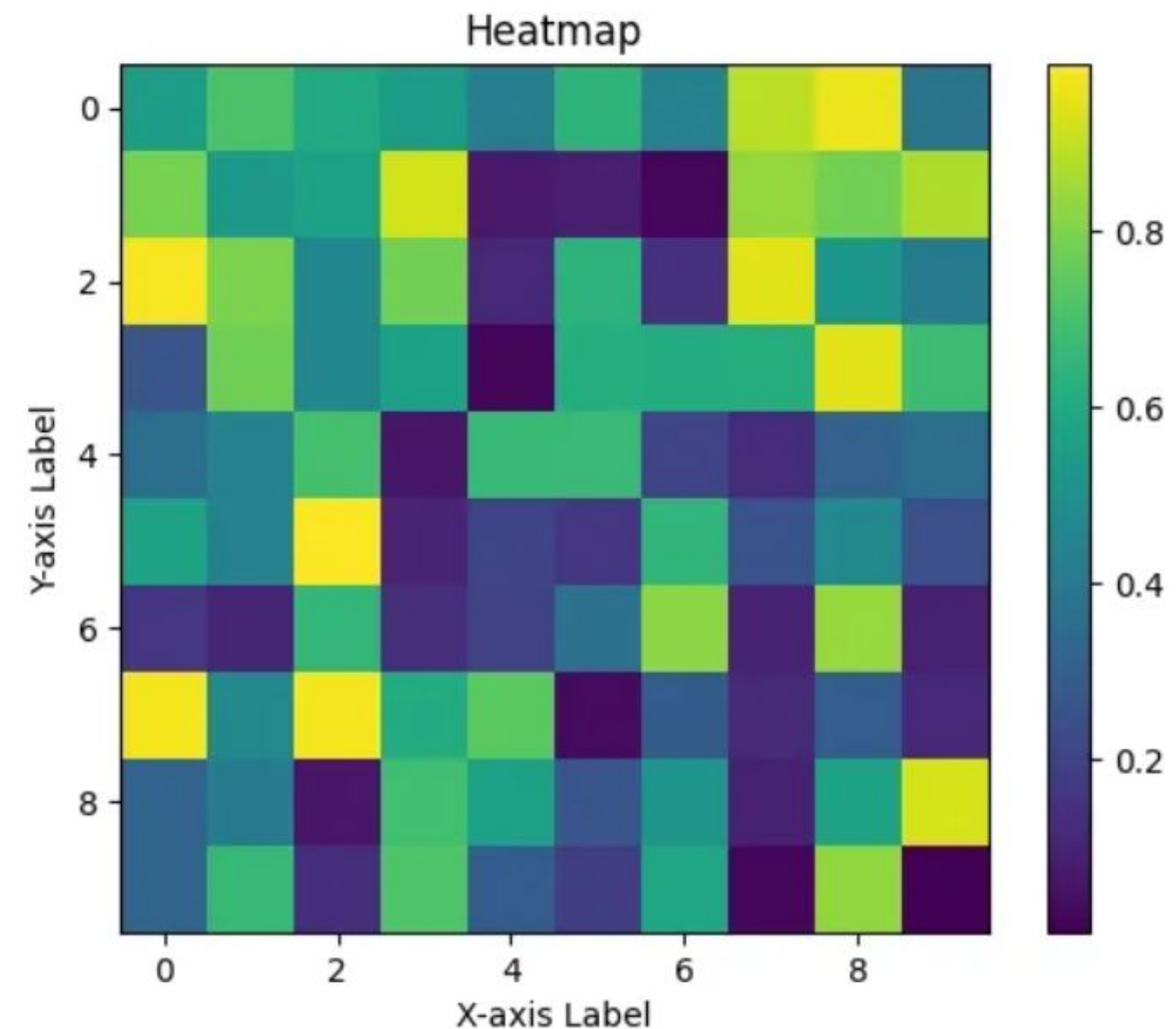
X: 2D array (data to display as an image or heatmap).

cmap: Sets the color map.

Example: This code creates a heatmap of random 10×10 data using imshow(). It uses 'viridis' color map and colorbar() adds a color scale.

```
import matplotlib.pyplot as plt
import numpy as np

np.random.seed(0)
data = np.random.rand(10, 10)
plt.imshow(data, cmap='viridis', interpolation='nearest')
plt.colorbar()
plt.xlabel('X-axis Label')
plt.ylabel('Y-axis Label')
plt.title('Heatmap')
plt.show()
```



Choosing the Right Library

LIBRARY	USE IT FOR	CORE OBJECT / INTERFACE
NumPy	Fast numeric arrays, matrix maths	ndarray
Pandas	Tabular data, cleaning, analysis	DataFrame, Series
SciPy	Statistics, optimisation, integration	scipy.stats, scipy.optimize
Matplotlib	Charts: line, bar, scatter, histogram	pyplot

Complementary, not competing — a typical data-science script imports all four.

Key Takeaways — Python Libraries & Tools

- 1 The stack is layered: NumPy arrays underpin SciPy, Pandas, and everything above them.
- 2 Vectorised NumPy operations replace slow Python loops — think in arrays, not elements.
- 3 Pandas is where data preparation happens — read, inspect, filter, and fix missing values.
- 4 SciPy supplies textbook maths ready-made; Matplotlib turns every result into a labelled chart.

QUICK QUIZ

Review Questions

Q1 How does machine learning differ from traditional programming in where the rules come from?

Q3 What distinguishes supervised from unsupervised learning?

Q5 Which library and data structure would you use to load and filter a CSV file?

Q2 List the seven steps of the machine learning process in order.

Q4 Which ML type does AlphaGo use, and what feedback does it learn from?

Q6 Name two Pandas methods for handling missing values.

FURTHER STUDY

References & Further Reading

Book	Géron, A. — <i>Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow</i> , O'Reilly
Book	Müller, A. & Guido, S. — <i>Introduction to Machine Learning with Python</i> , O'Reilly
Book	McKinney, W. — <i>Python for Data Analysis</i> , O'Reilly (by the creator of Pandas)
Docs	numpy.org · pandas.pydata.org · scipy.org · matplotlib.org — official tutorials, free
Course	Andrew Ng — <i>Machine Learning Specialization</i> , Coursera (audit free)

Thank You